Global Exception Handling in Spring Boot

Complete notes with clear, readable Java programs, annotations, examples, and interview questions.

1. What is an Exception?

An exception is an unexpected event that interrupts the normal flow of a program.

```
int a = 10;
int b = 0;

int result = a / b;
```

This causes an **ArithmeticException** because division by zero is not allowed.

2. What is Global Exception Handling?

Global Exception Handling means handling exceptions from multiple controllers in one centralized class instead of writing try-catch blocks repeatedly in every controller.

Benefits: cleaner code, centralized handling, consistent API responses, easier maintenance, and appropriate HTTP status codes.

3. Important Annotations

@**ExceptionHandler** — handles a specific exception.

@**ControllerAdvice** — provides global controller-level exception handling.

@**RestControllerAdvice** — global exception handling for REST APIs; combines the behavior of @ControllerAdvice and @ResponseBody.

@**ResponseBody** — writes the return value directly into the HTTP response body.

4. @ExceptionHandler

@ExceptionHandler specifies which exception a particular method should handle.

```
@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<String> handleIllegalArgumentException(
IllegalArgumentException ex) {

return ResponseEntity
.badRequest()
.body(ex.getMessage());
}
```

Meaning: if **IllegalArgumentException** occurs, Spring can execute this handler method.

5. @ControllerAdvice

@ControllerAdvice is used to create shared exception-handling logic for controllers.

```
@ControllerAdvice
public class GlobalExceptionHandler {

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<String> handleException(
IllegalArgumentException ex) {

return ResponseEntity
.badRequest()
.body(ex.getMessage());
}
}
```

For REST APIs, **@RestControllerAdvice** is usually more convenient.

6. @RestControllerAdvice

@RestControllerAdvice is commonly used for global exception handling in Spring Boot REST APIs.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

It provides global controller advice and response-body behavior, so handler return values can be serialized as REST responses.

7. Difference Between the Annotations

@ExceptionHandler → Handles a particular exception.

@ControllerAdvice → Provides global controller advice.

@RestControllerAdvice → Provides global exception handling for REST APIs.

@ResponseBody → Writes the return value into the HTTP response body.

8. Complete Student Example

Service method that validates the student name before saving:

```
public StudentData createStudent(StudentData student) {

if (student.getName() == null) {
throw new IllegalArgumentException(
"Student name cannot be null");
}

return studentRepo.save(student);
}
```

If the request contains **"name": null**, the service throws IllegalArgumentException and the exception can be handled by the global handler.

9. Global Exception Handler Example

```
@RestControllerAdvice
public class GlobalExceptionHandler {

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<String> handleIllegalArgumentException(
IllegalArgumentException ex) {

return ResponseEntity
.badRequest()
.body(ex.getMessage());
}
}
```

The client receives **HTTP 400 Bad Request** with the message: **Student name cannot be null**.

10. Custom Exception

For business-specific errors, it is better to create custom exceptions instead of using generic exceptions everywhere.

```
public class StudentNotFoundException
extends RuntimeException {

public StudentNotFoundException(String message) {
super(message);
}
}
```

Service example:

```
public StudentData getStudentById(Integer id) {

return studentRepo.findById(id)
.orElseThrow(() ->
new StudentNotFoundException(
"Student not found with id: " + id
)
);
}
```

Handler:

```
@ExceptionHandler(StudentNotFoundException.class)
public ResponseEntity<String> handleStudentNotFound(
StudentNotFoundException ex) {

return ResponseEntity
.status(HttpStatus.NOT_FOUND)
.body(ex.getMessage());
}
```

11. Proper Error Response

Instead of returning only a string, real APIs commonly return a structured JSON error object.

```
public class ErrorResponse {

private String message;
private int status;
private LocalDateTime timestamp;

public ErrorResponse(String message, int status) {
this.message = message;
this.status = status;
this.timestamp = LocalDateTime.now();
}

// getters and setters
}

@ExceptionHandler(StudentNotFoundException.class)
public ResponseEntity<ErrorResponse> handleStudentNotFound(
StudentNotFoundException ex) {

ErrorResponse error = new ErrorResponse(
ex.getMessage(),
HttpStatus.NOT_FOUND.value()
);

return ResponseEntity
.status(HttpStatus.NOT_FOUND)
.body(error);
}
```

Example JSON response:

```
{
"message": "Student not found with id: 100",
"status": 404,
"timestamp": "2026-09-09T20:30:15"
}
```

12. Handling Multiple Exceptions

```
@RestControllerAdvice
public class GlobalExceptionHandler {

@ExceptionHandler(StudentNotFoundException.class)
public ResponseEntity<ErrorResponse> handleStudentNotFound(
StudentNotFoundException ex) {

ErrorResponse error =
new ErrorResponse(ex.getMessage(), 404);

return ResponseEntity
.status(HttpStatus.NOT_FOUND)
.body(error);
}

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<ErrorResponse> handleIllegalArgument(
IllegalArgumentException ex) {

ErrorResponse error =
new ErrorResponse(ex.getMessage(), 400);

return ResponseEntity
.badRequest()
.body(error);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleGenericException(
Exception ex) {

ErrorResponse error =
new ErrorResponse(
"Something went wrong", 500);

return ResponseEntity
.status(HttpStatus.INTERNAL_SERVER_ERROR)
.body(error);
}
}
```

The generic Exception handler is a fallback. In production, avoid exposing sensitive internal exception details to clients; log the actual exception on the server.

13. ResponseEntity

ResponseEntity lets you control the HTTP status, response body, and headers.

```
return ResponseEntity
.status(HttpStatus.NOT_FOUND)
.body(error);
```

This returns HTTP 404 with the error object as the response body.

14. null vs Empty vs Blank

null means no value, **""** is an empty string, and **" "** contains whitespace.

```
if (student.getName() == null ||
student.getName().isBlank()) {

throw new IllegalArgumentException(
"Student name cannot be blank");
}
```

Java 11+ **isBlank()** rejects empty strings and strings containing only whitespace. For REST DTO validation, **@NotBlank** is also useful.

15. Validation Example

```
public class StudentData {

@NotBlank(message = "Student name is required")
private String name;

@Min(value = 18,
message = "Age must be at least 18")
private Integer age;

// getters and setters
}

@PostMapping
public StudentData createStudent(
@Valid @RequestBody StudentData student) {

return studentService.createStudent(student);
}
```

Validation happens before the service method is executed when the request violates the declared constraints. Validation exceptions can also be handled centrally.

16. Typical Flow

```
Client
|
v
Controller
|
v
Service
|
v
Exception occurs
|
v
@RestControllerAdvice
|
v
@ExceptionHandler
|
v
ErrorResponse + HTTP Status
|
v
Client
```

17. Common HTTP Status Codes

400 BAD_REQUEST → Invalid input or validation error.

401 UNAUTHORIZED → Authentication is required.

403 FORBIDDEN → User is authenticated but does not have permission.

404 NOT_FOUND → Requested resource does not exist.

409 CONFLICT → Request conflicts with current resource state.

500 INTERNAL_SERVER_ERROR → Unexpected server-side error.

18. Interview Questions & Answers

Q: What is Global Exception Handling?
A: It is a mechanism in Spring Boot to handle exceptions centrally for multiple controllers, commonly using @RestControllerAdvice and @ExceptionHandler.

Q: What is @ExceptionHandler?
A: It defines a method that handles a specific exception.

Q: What is @RestControllerAdvice?
A: It provides global exception handling for REST controllers and response-body behavior.

Q: Why use Global Exception Handling?
A: To avoid duplicate code, keep controllers clean, return consistent error responses, and use appropriate HTTP status codes.

Q: Can we have multiple @ExceptionHandler methods?
A: Yes. We can define handlers for different exception types.

Q: Can @ExceptionHandler be used without @RestControllerAdvice?
A: Yes. It can be placed inside a controller for controller-specific handling, or inside an advice class for global handling.

19. Quick Revision

```
@RestControllerAdvice
|
+--> Global Exception Handler Class
|
+--> @ExceptionHandler
|
+--> Specific Exception
|
+--> ErrorResponse
|
+--> HTTP Status
```

Best interview sentence: @RestControllerAdvice creates a centralized global exception-handling class, while @ExceptionHandler defines how a particular exception should be handled and what response should be returned to the client.